

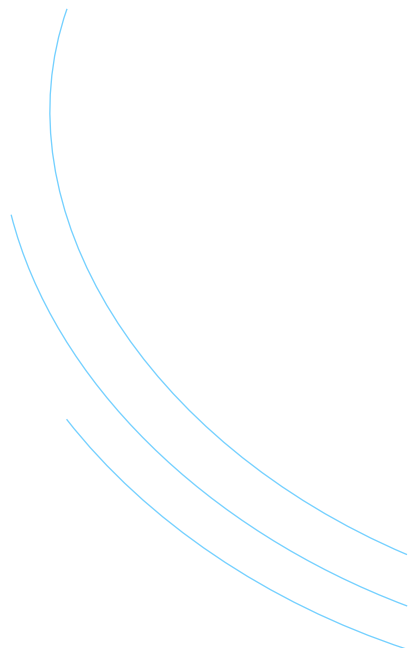
2025

Introdução de CSS - Cascading Style Sheets

Prof. M. Sc. Felipe Ivo da Silva
Coautor: Breno Nascimento Lopes (IFSP)
Baseado no Prof. Dr. Sérgio Luisir Díscola Junior



SUMÁRIO

- 1. Estrutura e formas de aplicar CSS**
 - 2. Seletores**
 - 3. Cores e background**
 - 4. Visibilidade**
 - 5. Modelo de caixa**
 - 6. Dimensionamento e posicionamento de elementos**
 - 7. Flex-box**
- 

Abordagens em cada tópico

Fundamentos

1. Estrutura e formas de aplicar CSS

Antes de estilizar, é essencial saber como o CSS entra no HTML.

2. Cores e background

Comece com estilos visuais simples para entender como o CSS afeta a aparência.

3. Seletores

Depois de entender o impacto visual, aprenda a selecionar os elementos certos.

4. Modelo de caixa (Box Model)

Agora que você estiliza elementos, é hora de entender sua estrutura física.

5. Dimensionamento e posicionamento de elementos

Com a caixa definida, você aprende a controlar tamanho e posição na página.

6. Display e visibilidade

Por fim, controle como os elementos aparecem ou se comportam no layout.

7. Flex-box

Com os fundamentos de layout dominados, é hora de usar o Flexbox para alinhar, distribuir e organizar elementos de forma eficiente e responsiva em uma única dimensão.

Revisão CSS

Cascading Style Sheets



Tag html <style>

CSS

Para mudarmos a aparência precisarmos chamar a tag e para isso tem três formas: Colocando o nome da tag (Todos os elementos de mesma tag mudarão) ex :

```
h1 {  
  color: ■darkblue;  
  font-size: 32px;  
  text-align: center;  
}
```

```
.imagemZebra {  
  width: 300px;  
  border: 2px solid ■black;  
}
```

```
#nascimento {  
  padding: 8px;  
  border: 1px solid ■#ccc;  
  background-color: ■#f9f9f9;  
}
```

Ao usar o atributo **class**, podemos aplicar estilos ou comportamentos a diversos elementos, já que ele é reutilizável. Já o atributo **id** deve ser usado em elementos únicos na página, como campos específicos de formulários, pois seu valor não pode se repetir.

Em ambos os casos, somos nós que definimos os nomes. Por exemplo: class="imagemZebra" ou id="nascimento", escolhidos conforme a função.

CLASS -

ID - <input type="date" id="nascimento" name="nascimento">

Tag style

CSS

```
h1 {  
  color:  darkblue;  
  font-size: 32px;  
  text-align: center;  
}
```

H1 é escrito apenas h1 e irá alterar todos os h1 da página

```
.imagemZebra {  
  width: 300px;  
  border: 2px solid  black;  
}
```

A classe imagemZebra terá um ponto  antes do nome e aplica o estilo a todos os elementos com a classe imagemZebra.

```
#nascimento {  
  padding: 8px;  
  border: 1px solid  #ccc;  
  background-color:  #f9f9f9;  
}
```

O id nascimento terá um cerquilha  antes do nome e alterará apenas esse elemento.

O que é CSS

Cascading Style Sheets



Cascading Style Sheets

CSS

Folhas de Estilo em Cascata

HTML (Estrutura):

São as paredes, vigas, portas e janelas.

Define a **estrutura** e o **conteúdo** (títulos, parágrafos, imagens, links).

CSS (Estilo e Design):

É a pintura, o papel de parede, a decoração, o paisagismo.

Define a **aparência** e o **layout** (cores, fontes, espaçamentos, posicionamento).

Cascading Style Sheets

CSS

O CSS é usado para controlar TUDO relacionado à apresentação visual de um site:

Cores e Fundos: Cor do texto, cor de fundo, imagens de fundo.

Tipografia: Fontes, tamanhos, espessura, espaçamento entre letras e linhas.

Layout e Posicionamento: Onde os elementos ficam na página (header, menu, colunas, footer).

Espaçamento: Margens e preenchimentos internos (margin e padding).

Efeitos Visuais: Sombras, bordas arredondadas, transições e animações.

Design Responsivo: Adaptar o *layout* para diferentes tamanhos de tela (celular, tablet, desktop).

Cascading Style Sheets

CSS

Por que o CSS é importante? (Vantagens)

Seperação de Conteúdo e Design: O HTML cuida do conteúdo, o CSS do visual. Isso torna a manutenção muito mais fácil.

Consistência: Altere o estilo em todo o site mudando apenas um arquivo CSS.

Eficiência: Um único arquivo CSS pode estilizar múltiplas páginas HTML.

Flexibilidade: Permite criar *layouts* complexos e *designs* modernos.

Experiência do Usuário: Cria sites visualmente atraentes, profissionais e fáceis de navegar.

Estrutura e formas de aplicar CSS

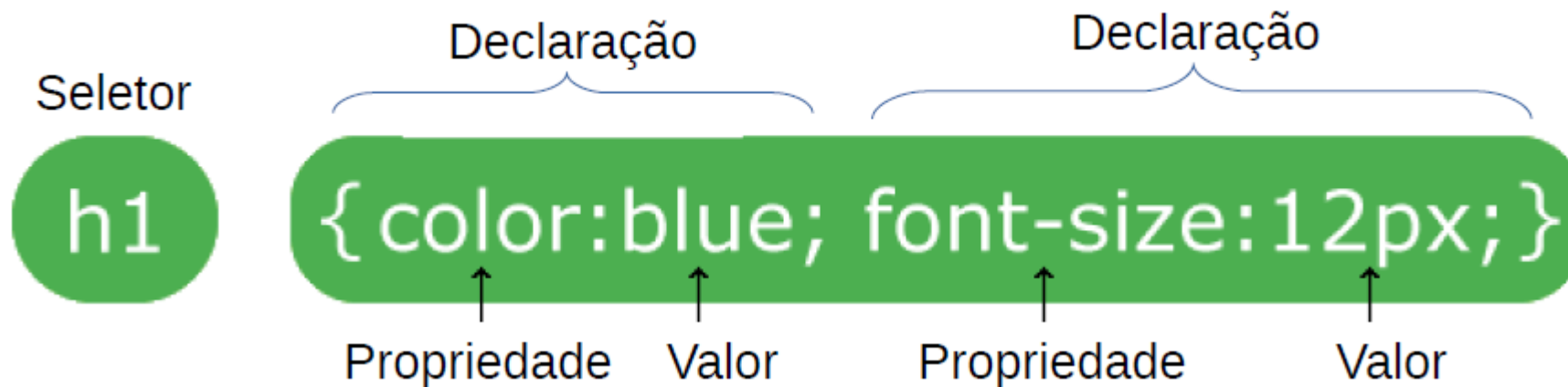
Cascading Style Sheets

```
26 .screen-reader-text:hover,  
27 .screen-reader-text:active,  
28 .screen-reader-text:focus {  
29     background-color: #f1f1f1;  
30     border-radius: 3px;  
31     box-shadow: 0 0 2px 2px rgba(0, 0, 0, 0.6);  
32     clip: auto !important;  
33     color: #21759b;  
34     display: block;  
35     font-size: 14px;  
36     font-size: 0.875rem;  
37     font-weight: bold;  
38     height: auto;  
39     left: 5px;  
40     line-height: normal;  
41     padding: 15px 23px 14px;  
42     text-decoration: none;  
43     top: 5px;  
44     width: auto;  
45     z-index: 100000; /* Above WP toolbar. */  
46 }  
47
```

Estrutura e formas de aplicar CSS

CSS

Sintaxe: do CSS é baseada em regras que contêm um seletor, que identifica os elementos HTML a serem estilizados, e um bloco de declarações entre chaves {}



- O seletor aponta para o elemento HTML que você deseja estilizar. Neste exemplo, os elementos h1;
- O bloco de declaração contém uma ou mais declarações separadas por ponto e vírgula.
- Cada declaração inclui um nome de propriedade CSS e um valor, separados por dois pontos. Neste exemplo: **color:blue;** e **font-size:12px;**
- Várias declarações CSS são separadas por ponto e vírgula e os blocos de declaração são cercados por chaves.

Cascading Style Sheets

CSS

A tag `<style>` é uma etiqueta HTML que serve para incorporar código CSS diretamente em um documento HTML.

```
1  <!DOCTYPE html>
2  <html>
3    <head>
4      <meta charset="utf-8">
5      <title>Sintaxe do CSS</title>
6      <style>
7        p {
8          color: ■ red;
9          text-align: center;
10       }
11     </style>
12   </head>
13   <body>
14     <p>Est página mostra como a sintaxe do CSS funciona!</p>
15     <p>Este parágrafo está estilizado com CSS.</p>
16   </body>
17 </html>
```

A tag `<style>` é usada para definir informações de estilo (CSS) para um documento.

Cascading Style Sheets

CSS

Seletor p

```
1  <!DOCTYPE html>
2  <html>
3    <head>
4      <meta charset="utf-8">
5      <title>Sintaxe do CSS</title>
6      <style>
7        p {
8          color: ■ red;
9          text-align: center;
10       }
11     </style>
12   </head>
13   <body>
14     <p>Est página mostra como a sintaxe do CSS funciona!</p>
15     <p>Este parágrafo está estilizado com CSS.</p>
16   </body>
17 </html>
```

Seletor p

→ Os seletores CSS são usados para "encontrar" (ou selecionar) os elementos HTML que você deseja estilizar.

Cascading Style Sheets

CSS

```
1  <!DOCTYPE html>
2  <html>
3      <head>
4          <meta charset="utf-8">
5          <title>Sintaxe do CSS</title>
6          <style>
7              p {
8                  color: ■ red;
9                  text-align: center;
10             }
11          </style>
12      </head>
13      <body>
14          <p>Est página mostra como a sintaxe do CSS funciona!</p>
15          <p>Este parágrafo está estilizado com CSS.</p>
16      </body>
17  </html>
```

Declaração:

par propriedade:valor, onde:

- *Propriedade* é o que desejamos estilizar;
- *Valor* é o valor do estilo que estamos aplicando.
- As declarações são separadas por ponto-e-vírgula.

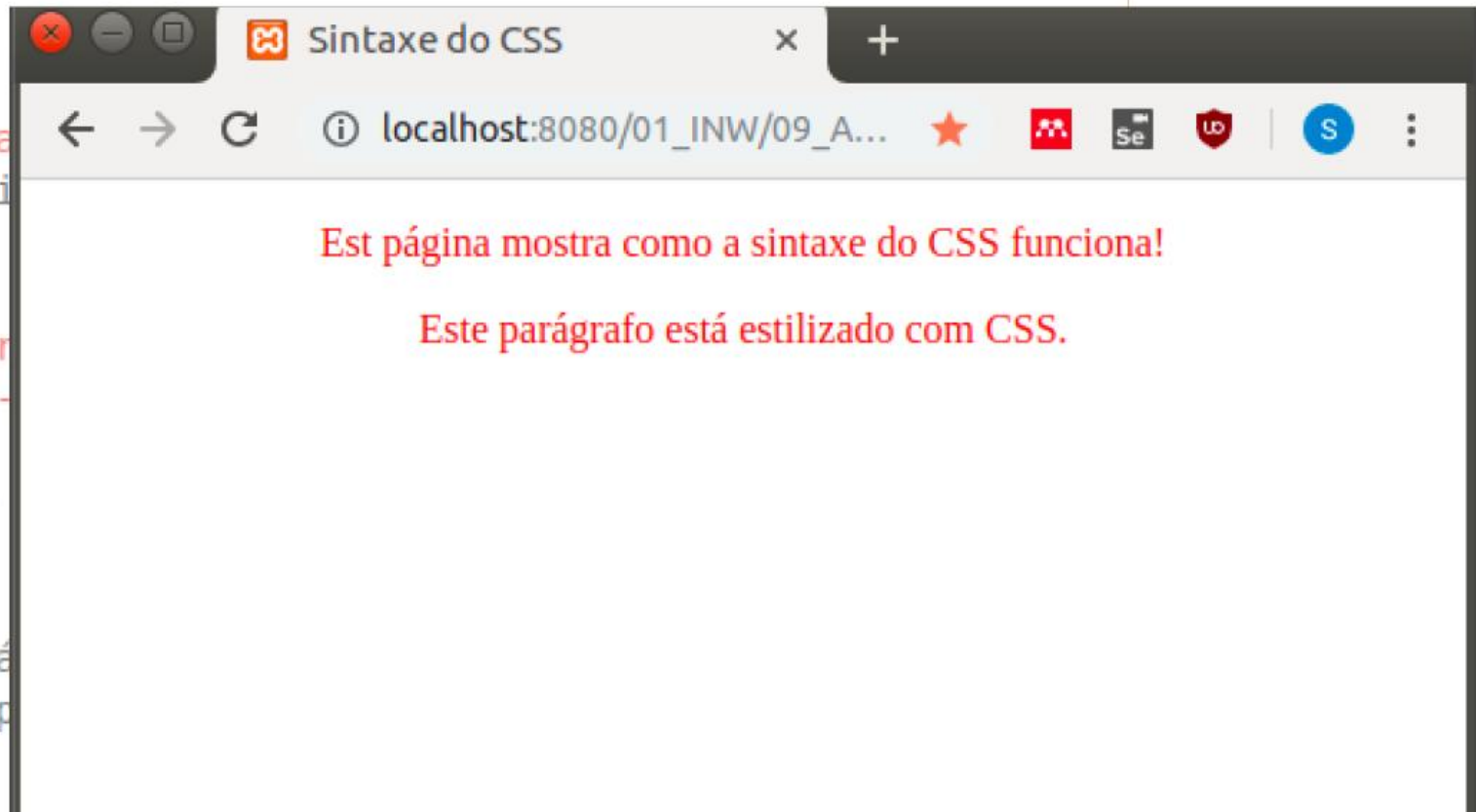
Cascading Style Sheets

CSS

Resultado

```
1  <!DOCTYPE html>
2  <html>
3    <head>
4      <meta char...
5      <title>Si...
6      <style>
7        p {
8          color: red;
9          text-align: center;
10         }
11      </style>
12    </head>
13    <body>
14      <p>Est pá...
15      <p>Este p...
16    </body>
17  </html>
```

Resultado



Cascading Style Sheets

CSS

Novo exemplo

```
1  <!DOCTYPE html>
2  <html>
3    <head>
4      <meta charset="utf-8">
5      <title>Segundo exemplo de CSS</title>
6      <style>
7        body {
8          background-color: blueviolet;
9        }
10       h1 {
11         color: white;
12         text-align: center;
13       }
14       p {
15         font-family: verdana;
16         font-size: 25px;
17       }
18     </style>
19   </head>
20   <body>
21     <h1>Múltiplas regras CSS</h1>
22     <p>Este é o parágrafo.</p>
23   </body>
24 </html>
```

Regra CSS-01

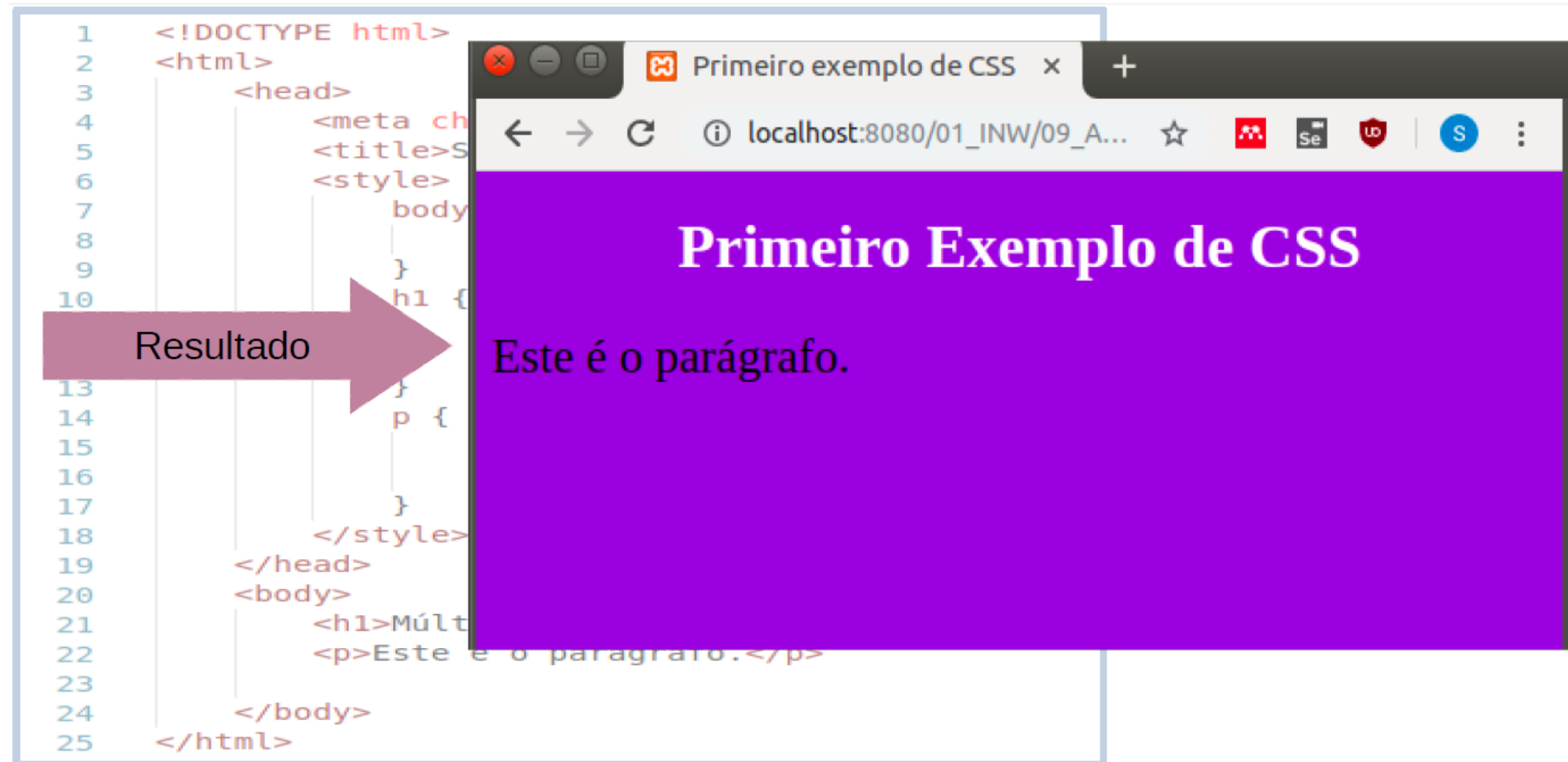
Regra CSS-02

Regra CSS-02

Cascading Style Sheets

CSS

Resultado



Seletores

Cascading Style Sheets

HTML

```
<!DOCTYPE html>
<html>
<head>
  <title>CSS: Seletores</title>
  <link rel="stylesheet" type="text/
css" href="style.css">
</head>
<body>
  <p class="corAzul corClaro"> Texto
simples</p>
  <p class="corAzul"> Texto simples 2
</p>
  <p> Texto simples 3</p>
</body>
</html>
```

CSS

```
.corAzul.corClaro{
  color: lightblue;
}
```



Seletores

CSS

- Para poder estilizar um determinado **elemento HTML** utilizando CSS, você precisa ter uma maneira de **selecionar esses elementos**.
- Na CSS a parte de uma regra de **estilo** que faz isso é chamado ***seletor***.
- Seletores CSS podem ser divididos em cinco categorias:
 - **Seletores simples** (a seleção é baseada nos atributo ***id***, ***name*** e ***class***);
 - **Pseudoclasses** (a seleção é baseada em estados);
 - **Pseudo-elementos** (seleciona e estiliza parte de um elemento);
 - **Seletores de atributo** (a seleção é baseada em um atributo ou seu valor).

Seletores simples baseados em elementos

CSS

O seletor de elemento seleciona elementos HTML com base no **nome do elemento**.

Exemplo:

```
1  <!DOCTYPE html>
2  <html>
3    <head>
4      <meta charset="utf-8">
5      <title>Sintaxe do CSS</title>
6      <style>
7        p {
8          color: red;
9          text-align: center;
10       }
11     </style>
12   </head>
13   <body>
14     <p>Est página mostra como a sintaxe do CSS funciona!</p>
15     <p>Este parágrafo está estilizado com CSS.</p>
16   </body>
17 </html>
```

Neste exemplo todos os elementos `<p>` são selecionados e as propriedades cor da fonte e alinhamento do texto são estilizados vermelho e centralizado, respectivamente.

Seletores simples baseados em id

CSS

- O seletor baseado em id usa este atributo para selecionar um elemento específico, já que os **id's** devem ser únicos dentro de uma página; portanto, o seletor de identificação é usado para selecionar um único elemento.
- Para escrever esse seletor use o símbolo **# seguido do id desejado**.
- **Exemplo:**

```
1  <!DOCTYPE html>
2  <html>
3    <head>
4      <meta charset="utf-8">
5      <title>Segundo exemplo de CSS</title>
6      <style>
7        #p2{
8          text-align: center;
9          color: darksalmon;
10         background-color: aqua;
11        }
12      </style>
13    </head>
14    <body>
15      <h1>Múltiplas regras CSS</h1>
16      <p id="p1">Primeiro parágrafo.</p>
17      <p id="p2">Segundo parágrafo.</p>
18    </body>
19  </html>
```

Neste exemplo somente o elemento HTML com id="p2" é selecionado. observe que o seletor consiste no símbolo # seguido do nome do id desejado, ou seja #p2.

Aqui o alinhamento do texto (text-align) é estilizado para o centro, a cor da fonte (color) como salmão escura, e a cor de fundo (background-color), como azul água.

Seletores simples baseados em id

CSS

Resultado:



Seletores simples baseados em classes

CSS

- O seletor baseado em **classe** usa o atributo **class** para selecionar um ou mais elementos.
- Para escrever esse seletor use o símbolo **.** **seguido da classe desejado.**
- **Exemplo:**

```
1  <!DOCTYPE html>
2  <html>
3    <head>
4      <meta charset="utf-8">
5      <title>Segundo exemplo de CSS</title>
6      <style>
7        .teste{
8          text-align: center;
9          color: darksalmon;
10         background-color: aqua;
11        }
12      </style>
13    </head>
14    <body>
15      <h1 class="teste">Múltiplas regras CSS</h1>
16      <p id="p1">Primeiro parágrafo.</p>
17      <p class="teste">Segundo parágrafo.</p>
18    </body>
19  </html>
```

Neste exemplo todos os elementos HTML com class="teste" são selecionados.

observe que o seletor consiste no símbolo . seguido do nome da classe desejada, ou seja .teste

Aqui o alinhamento do texto (text-align) é estilizado para o centro, a cor da fonte (color) como salmão escura, e a cor de fundo (background-color), como azul água.

Seletores simples baseados em id

CSS

Resultado:



Seletores simples baseados em classes

CSS

- Você também pode especificar que apenas elementos HTML específicos sejam afetados por uma classe.
- Exemplo:

```
1  <!DOCTYPE html>
2  <html>
3    <head>
4      <meta charset="utf-8">
5      <title>Segundo exemplo de CSS</title>
6      <style>
7        p.teste{
8          text-align: center;
9          color: darksalmon;
10         background-color: aqua;
11        }
12      </style>
13    </head>
14    <body>
15      <h1 class="teste">Múltiplas regras CSS</h1>
16      <p id="p1">Primeiro parágrafo.</p>
17      <p class="teste">Segundo parágrafo.</p>
18    </body>
19  </html>
```

Neste exemplo todos os elementos HTML parágrafo com class="teste" são selecionados. observe que o seletor consiste no símbolo p.teste

Seletores Combinados

CSS

Um seletor de CSS pode conter mais de um seletor simples. Existem quatro combinadores diferentes no CSS:

```
div p {  
  background-color: yellow  
}
```

Seletor descendente (espaço): seleciona todos os elementos <p> dentro dos elementos <div>

```
div > p {  
  background-color: red;  
}
```

Seletor filho (>): seleciona todos os elementos <p> filhos de um elemento <div>.

```
div + p {  
  background-color: blue;  
}
```

Seletor irmão adjacente (+): O exemplo a seguir seleciona todos os elementos <p> que aparecem logo após os elementos <div>.

```
div ~ p {  
  background-color: pink;  
}
```

Seletor geral de irmãos (~): O exemplo a seguir seleciona todos os elementos <p> irmãos dos elementos <div>

Seletores *pseudoclasses*

CSS

- Uma **pseudo-classe** é usada para definir um estado especial de um elemento. Por exemplo, ele pode ser usado para:
 - Estilizar um elemento quando um usuário passar o mouse sobre ele
 - Estilizar links visitados e não visitados de maneira diferente
 - Estilizar um elemento quando ele receber foco

Seletor	Exemplo	Descrição
:active	a:active	Seleciona o link ativo
:checked	input:checked	Seleciona todo elemento <input> assinalado
:disabled	input:disabled	Seleciona todo elemento <input> desabilitado
:first-child	p:first-child	Seleciona todos os elementos <p> que são o primeiro filho de seu pai

Seletores pseudo-elementos

CSS

- Um **pseudo-elemento** CSS é usado para estilizar partes especificadas de um elemento. Ele pode ser usado para:
 - Estilizar a primeira letra ou linha de um elemento;
 - Inserir conteúdo antes ou depois do conteúdo de um elemento.

Seletor	Exemplo	Descrição do exemplo
::after	p::after	Inserir algo após o conteúdo de cada elemento <p>
::before	p::before	Inserir algo antes do conteúdo de cada elemento <p>
::first-letter	p::first-letter	Seleciona a primeira letra de cada elemento <p>
::first-line	p::first-line	Seleciona a primeira linha de cada elemento <p>
::selection	p::selection	Seleciona a parte de um elemento que é selecionado por um usuário

Seletores de atributo

CSS

O seletor **[attribute]** é usado para selecionar elementos com um atributo específico.

Seletor	Exemplo	Descrição do exemplo
[attribute]	[target]	Seleciona todos os elementos com um atributo "target"
[attribute=value]	[target=_blank]	Seleciona todos os elementos com target = "_ blank"
[attribute~=value]	[title~=flor]	Seleciona todos os elementos com um atributo <u>title</u> que contém a palavra "flor"
[attribute =value]	[lang =en]	Seleciona todos os elementos com um valor de atributo <u>lang</u> começando com "en"
[attribute^=value]	a[href^="https"]	Seleciona todos os elementos <a> cujo valor do atributo <u>href</u> começa com "https"
[attribute\$=value]	a[href\$=".pdf"]	Seleciona todos os elementos <a> cujo valor do atributo <u>href</u> termina com ".pdf"
[attribute*=value]	a[href*="w3schools"]	Seleciona todos os elementos <a> cujo valor do atributo <u>href</u> contém a substring "w3schools"

Cores e background

...



Cores e background

Color

A propriedade color define a **cor do texto** dentro de um elemento.

Exemplo:

```
color: red;
```

Você pode usar:

- **Nomes de cores:** red, blue, green
- **Hexadecimal:** #ff0000
- **RGB:** rgb(255, 0, 0)
- **HSL:** hsl(0, 100%, 50%)
- **Transparência:** rgba(255, 0, 0, 0.5)

Essa propriedade **não afeta o fundo**, apenas o conteúdo textual.

Cores e background

background (plano de fundo)

A propriedade background controla o **fundo do elemento**. Pode incluir cor, imagem, posição, repetição e mais.

Exemplo:

```
background: yellow;
```

Exemplo com imagem:

```
background: url('imagem.jpg') no-repeat center center; background-size: cover;
```

Você pode usar:

- background-color: só a cor de fundo
- background-image: imagem de fundo
- background-repeat: repetir ou não a imagem
- background-position: onde a imagem aparece
- background-size: como ela se ajusta ao elemento

Cores e background

CSS

A propriedade **background-color** especifica a cor de fundo de um elemento.

Exemplo:

- **background-color:** cor de fundo do elemento Selecionado.

- **opacity:** especifica a opacidade de um elemento. Varia de 0 a 1 e quanto menor, mais transparente a cor de fundo.

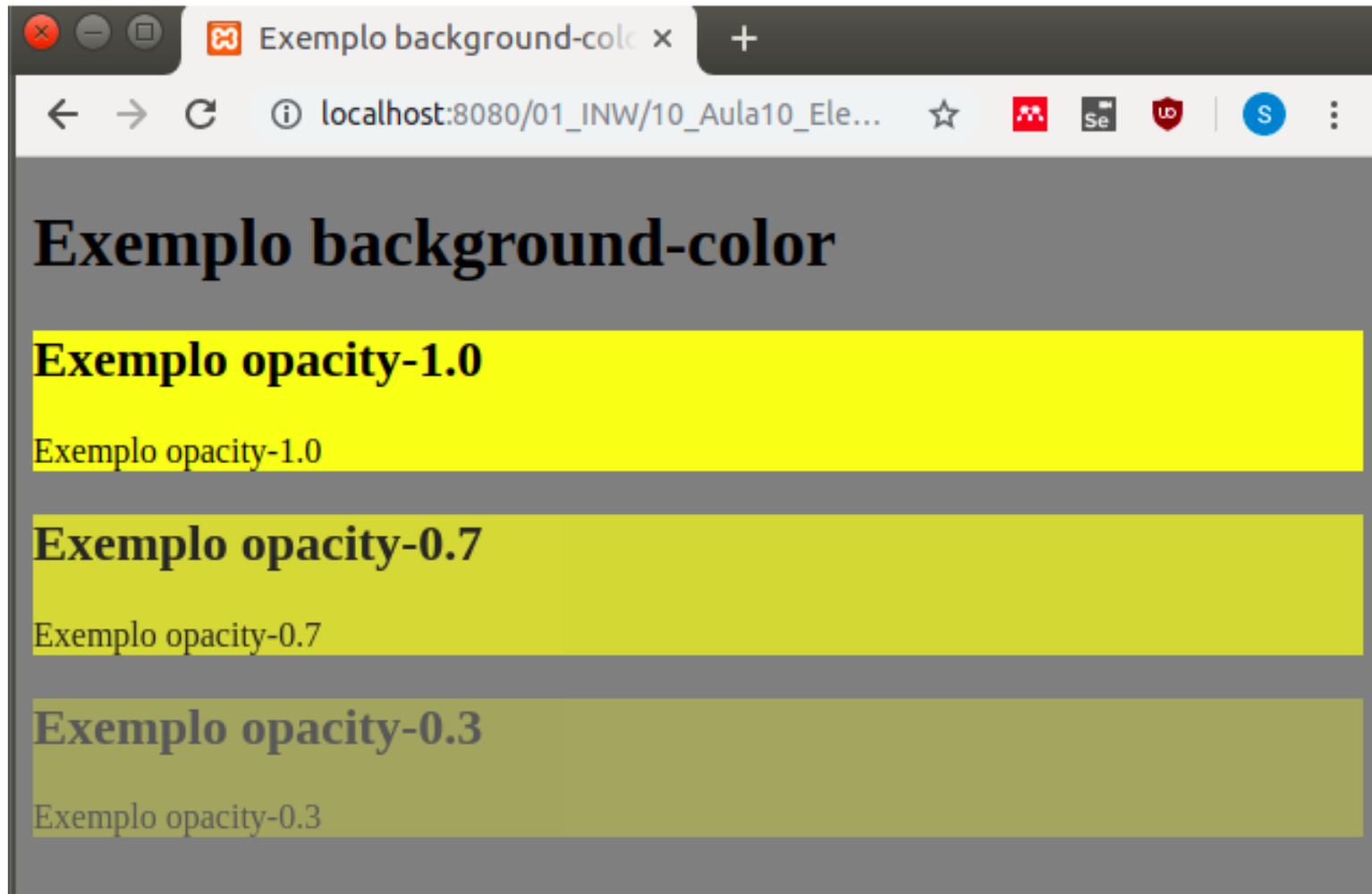
```
1  <!DOCTYPE html>
2  <html>
3    <head>
4      <meta charset="utf-8">
5      <style>
6        body{
7          background-color: gray;
8        }
9        #div1{
10         background-color: yellow;
11         opacity: 1.0;
12       }
13       #div2{
14         background-color: yellow;
15         opacity: 0.7;
16       }
17       #div3{
18         background-color: yellow;
19         opacity: 0.3;
20       }
21     </style>
22     <title>Exemplo background-color</title>
23  </head>
```

```
24  <body>
25    <h1>Exemplo background-color</h1>
26    <div id="div1">
27      <h2>Exemplo opacity-1.0</h2>
28      <p>Exemplo opacity-1.0 </p>
29    </div>
30    <div id="div2">
31      <h2>Exemplo opacity-0.7</h2>
32      <p>Exemplo opacity-0.7</p>
33    </div>
34    <div id="div3">
35      <h2>Exemplo opacity-0.3</h2>
36      <p>Exemplo opacity-0.3</p>
37    </div>
38  </body>
39 </html>
40
```

Cores e background

CSS

Resultado:



Cores e background

CSS

A propriedade **background-color** especifica uma imagem de fundo de um elemento. Por default, ela é repetida por toda a página. **Exemplo:**

- **background-image:** imagem de fundo do elemento selecionado.

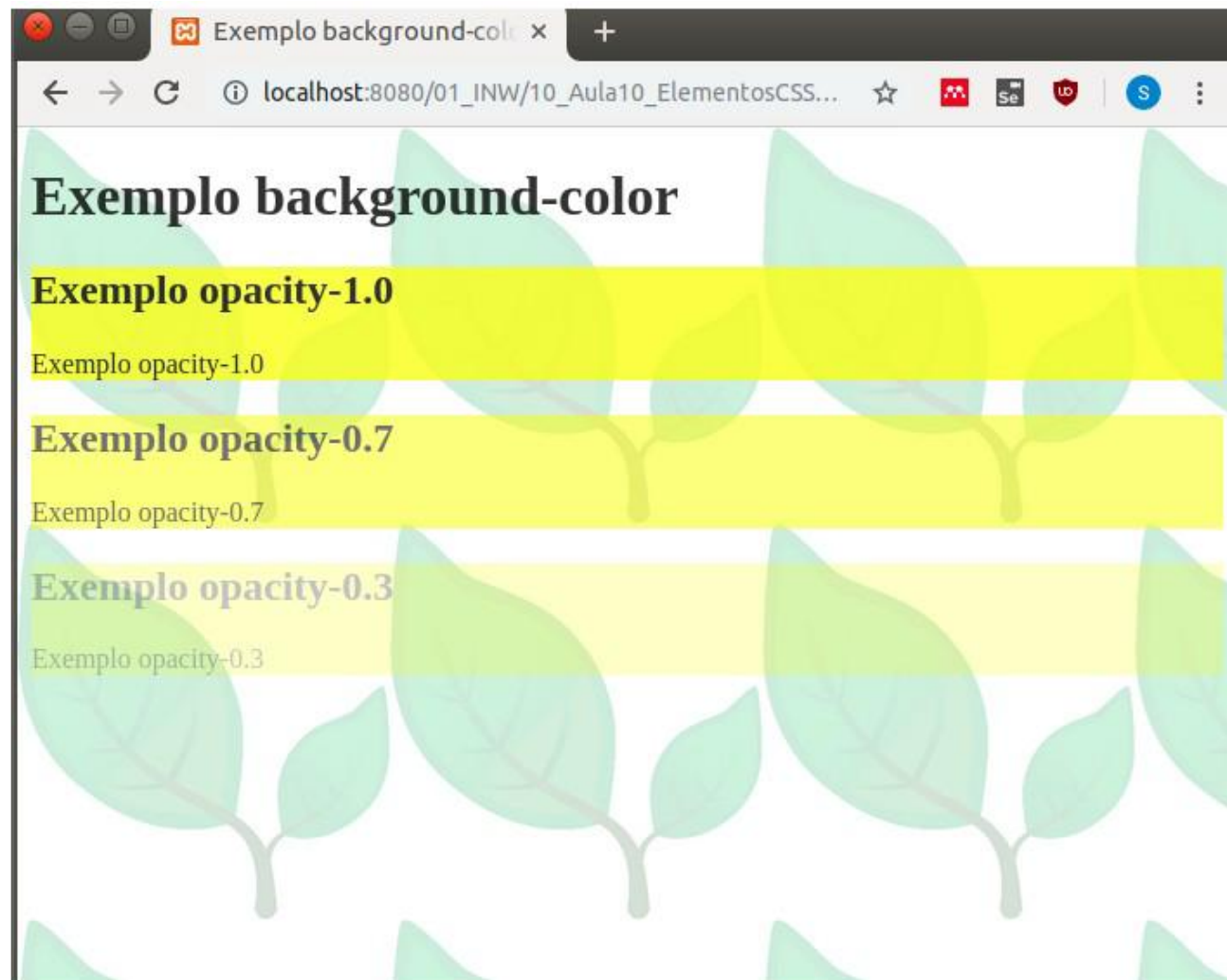
```
1  <!DOCTYPE html>
2  <html>
3    <head>
4      <meta charset="utf-8">
5      <style>
6        body{
7          background-image:url("01_imgs/folha.png");
8          opacity:0.8;
9        }
10       #div1{
11         background-color: yellow;
12         opacity: 1.0;
13       }
14       #div2{
15         background-color: yellow;
16         opacity: 0.7;
17       }
18       #div3{
19         background-color: yellow;
20         opacity: 0.3;
21       }
22     </style>
23     <title>Exemplo background-color</title>
24  </head>
```

```
25  <body>
26    <h1>Exemplo background-color</h1>
27    <div id="div1">
28      <h2>Exemplo opacity-1.0</h2>
29      <p>Exemplo opacity-1.0 </p>
30    </div>
31    <div id="div2">
32      <h2>Exemplo opacity-0.7</h2>
33      <p>Exemplo opacity-0.7</p>
34    </div>
35    <div id="div3">
36      <h2>Exemplo opacity-0.3</h2>
37      <p>Exemplo opacity-0.3</p>
38    </div>
39  </body>
40 </html>
```

Cores e background

CSS

Resultado:



Imagens, por padrão,
são inseridas uma após
a outra.

Cores e background

CSS

Por padrão, a propriedade **background-image** repete uma imagem horizontal e verticalmente. Algumas imagens devem ser repetidas apenas na horizontal ou na vertical. Para isso, usamos a propriedade **background-repeat**.

```
1  <!DOCTYPE html>
2  <html>
3    <head>
4      <meta charset="utf-8">
5      <style>
6        #div1>h2{
7          color: white;
8        }
9        #div1{
10         background-image: url("01_imgs/gradiente1.png");
11       }
12       #div2{
13         background-image: url("01_imgs/gradiente2.png");
14         background-repeat: repeat-x;
15       }
16       #div3{
17         background-image: url("01_imgs/gradiente3.png");
18         background-repeat: repeat-y;
19         background-size: 250px 50%;
20       }
21     </style>
22     <title>Exemplo background-color</title>
23  </head>
```

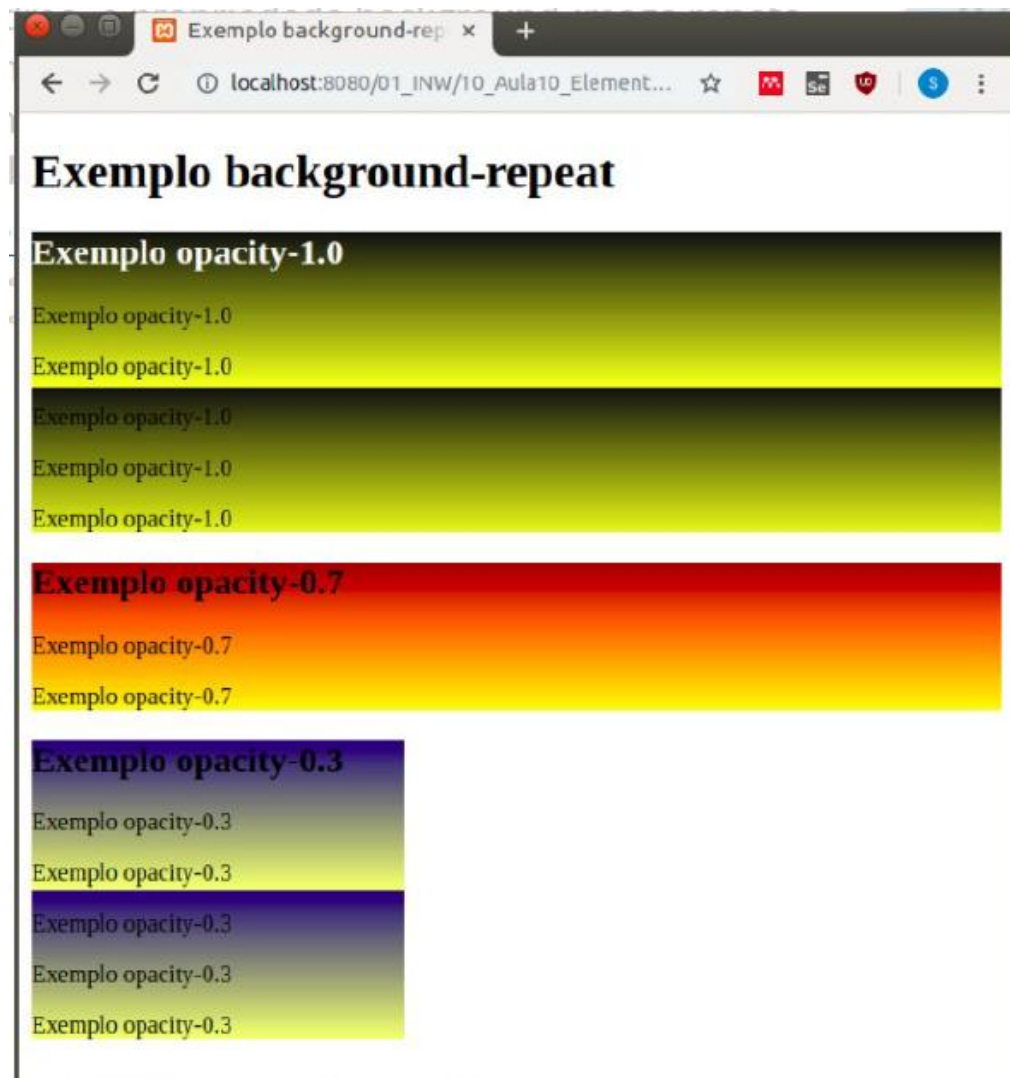
Background-repeat: altera como o modo de repetição de imagens é configurado. Explicação por valor:

- **repeat-x:** repete a imagem somente no eixo-x;
- **repeat-y:** repete a imagem somente no eixo-y;
- **no-repeat:** a imagem não é repetida.

Cores e background

CSS

Resultado:



background-repeat: altera como o modo de repetição de imagens é configurado. Explicação por valor:

- repeat-x: repete a imagem somente no eixo-x;
- repeat-y: repete a imagem somente no eixo-y;
- no-repeat: a imagem não é repetida.

Repetição da dir → esq / cima → baixo

Repetição no eixo x

Repetição no eixo y

```
26 <div id="div1">
27 <h2>Exemplo opacity-1.0</h2>
28 <p>Exemplo opacity-1.0 </p>
29 <p>Exemplo opacity-1.0 </p>
30 <p>Exemplo opacity-1.0 </p>
31 <p>Exemplo opacity-1.0 </p>
32 <p>Exemplo opacity-1.0 </p>
33 </div>
34 <div id="div2">
35 <h2>Exemplo opacity-0.7</h2>
36 <p>Exemplo opacity-0.7</p>
37 <p>Exemplo opacity-0.7</p>
38 <p>Exemplo opacity-0.7</p>
39 </div>
40 <div id="div3">
41 <h2>Exemplo opacity-0.3</h2>
42 <p>Exemplo opacity-0.3</p>
43 <p>Exemplo opacity-0.3</p>
44 <p>Exemplo opacity-0.3</p>
45 <p>Exemplo opacity-0.3</p>
46 <p>Exemplo opacity-0.3</p>
47 </div>
48 </body>
49 </html>
```


Cores e background

CSS

A propriedade **background-position** é usada para especificar a posição da imagem de fundo.

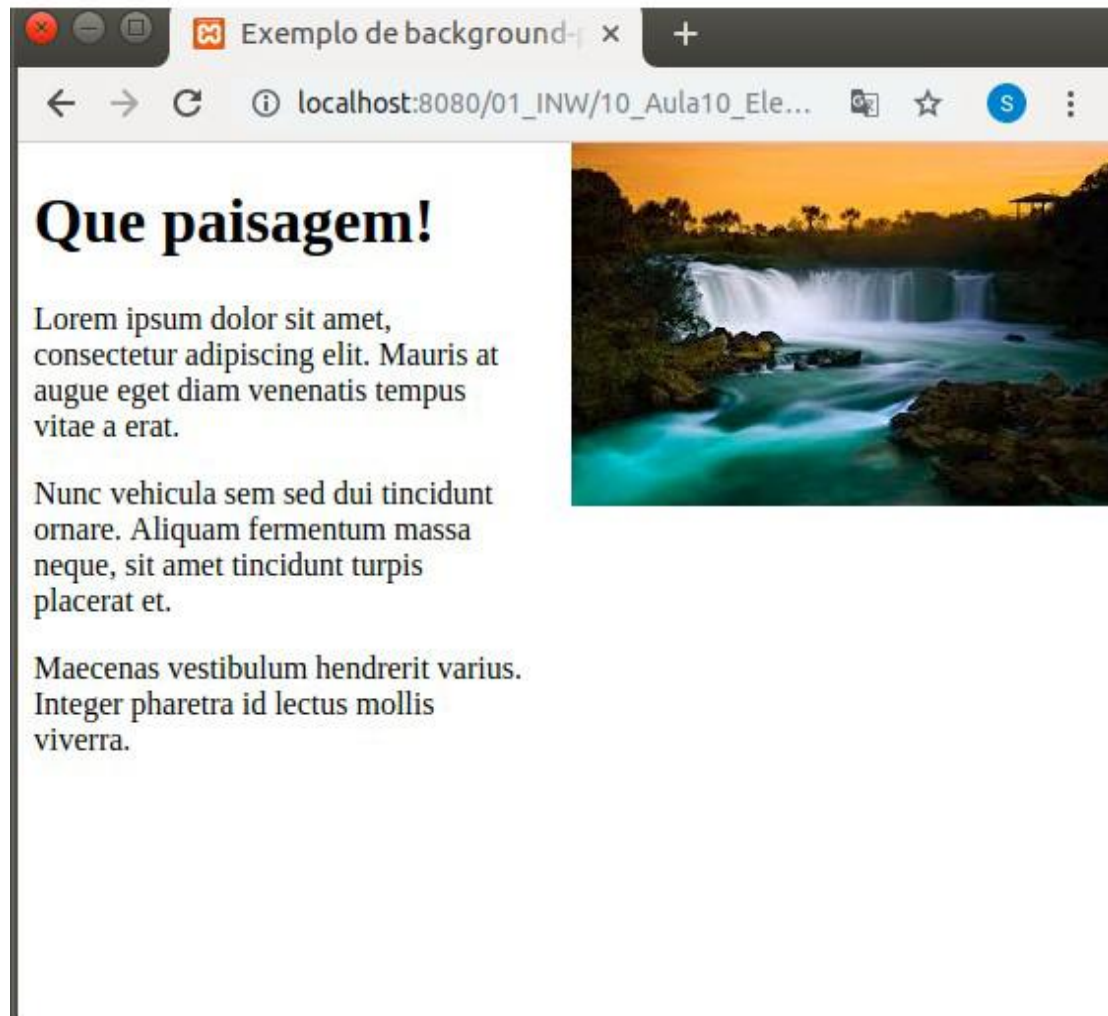
```
1  <!DOCTYPE html>
2  <html>
3    <head>
4      <meta charset="utf-8">
5      <title>Exemplo de background-position</title>
6      <style>
7        body {
8          background-image: url("01_imgs/paisagem.jpeg");
9          ;
10         background-repeat: no-repeat;
11         background-position: right top;
12         margin-right: 300px;
13       }
14     </style>
15   </head>
16   <body>
17     <h1>Que paisagem!</h1>
18     <p>Lorem ipsum dolor sit amet, consectetur adipiscing
19     elit. Mauris at augue eget diam venenatis tempus
20     vitae a erat.</p>
21     <p>Nunc vehicula sem sed dui tincidunt ornare.
22     Aliquam fermentum massa neque, sit amet tincidunt
23     turpis placerat et. </p>
24     <p>Maecenas vestibulum hendrerit varius. Integer
25     pharetra id lectus mollis viverra.</p>
26   </body>
27 </html>
```

- background-position: coloca a imagem de fundo no topo direito.

Cores e background

CSS

Resultado:



**Imagem no campo superior
direito (right top)**

Cores e background

CSS

A propriedade de anexo em segundo plano especifica se a imagem de segundo plano deve ser rolada ou não com o restante da página:

```
1  <!DOCTYPE html>
2  <html>
3    <head>
4      <meta charset="utf-8">
5      <title>Exemplo de background-attachment</title>
6      <style>
7        body {
8          background-image: url("01_imgs/paisagem.jpeg");
9          ;
10         background-repeat: no-repeat;
11         background-position: right top;
12         background-attachment: scroll;
13         margin-right: 300px;
14       }
15     </style>
16   </head>
17   <body>
18     <h1>Que paisagem!</h1>
19     <p>Lorem ipsum dolor sit amet, consectetur adipiscing elit. Mauris at augue eget diam venenatis tempus vitae a erat.</p>
20     <p>Nunc vehicula sem sed dui tincidunt ornare. Aliquam fermentum massa neque, sit amet tincidunt turpis placerat et. </p>
21     <p>Maecenas vestibulum hendrerit varius. Integer pharetra id lectus mollis viverra.</p>
22   </body>
23 </html>
```

A barra de rolagem surge e a imagem se movimenta.



Visibilidade

...



Visibilidade

...

A propriedade `visibility` controla **se o elemento está visível ou invisível**, mas **sem removê-lo do layout**.

Principais valores:

- `visible`: o padrão — o elemento aparece normalmente.
- `hidden`: o elemento **fica invisível**, mas **ainda ocupa espaço** na página.

Importante: `visibility: hidden` **mantém o espaço reservado** para o elemento, diferente de `display: none`.

Propriedade	Elemento aparece?	Ocupa espaço?	Pode ser animado?
<code>display: none</code>	✗ Não	✗ Não	✗ Não
<code>visibility: hidden</code>	✗ Não	✓ Sim	✓ Sim
<code>display: block/inline/etc</code>	✓ Sim	✓ Sim	✓ Sim

Visibilidade

...

html

Copy

```
<div class="box box1">Caixa 1</div>
<div class="box box2">Caixa 2</div>
<div class="box box3">Caixa 3</div>

<button onclick="usarVisibility()">Usar visibility: hidden</button>
<button onclick="usarDisplay()">Usar display: none</button>
<button onclick="resetar()">Resetar</button>
```

css

```
.box {
  display: inline-block;
  width: 100px;
  height: 100px;
  margin: 10px;
  text-align: center;
  line-height: 100px;
  color: white;
  font-weight: bold;
}

.box1 { background: red; }
.box2 { background: blue; }
.box3 { background: green; }
```

Visibilidade

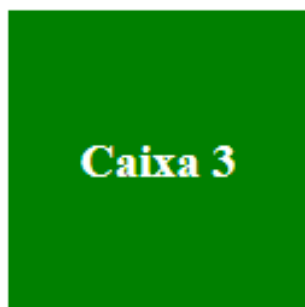
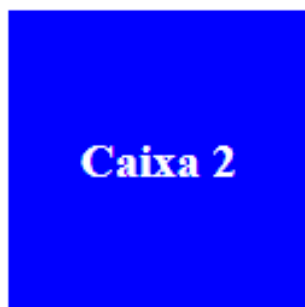
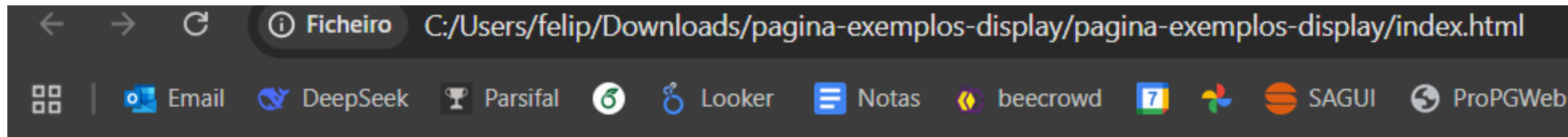
...

```
javascript
```

```
function usarVisibilidade() {  
    document.querySelector('.box2').style.visibility = 'hidden';  
}  
  
function usarDisplay() {  
    document.querySelector('.box2').style.display = 'none';  
}  
  
function resetar() {  
    document.querySelector('.box2').style.visibility = 'visible';  
    document.querySelector('.box2').style.display = 'inline-block';  
}
```

Visibilidade

Fica assim



Usar visibility: hidden

Usar display: none

Resetar

BOX MODEL

Modelo de caixa



Modelo de caixa

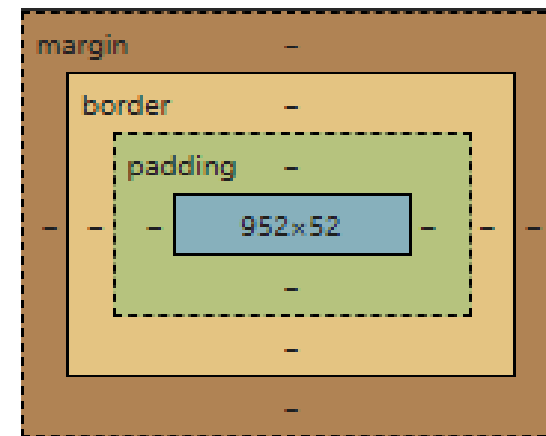
CSS

O modelo de caixa do CSS é um conceito fundamental para entender como os elementos HTML são renderizados na página. Cada elemento é tratado como uma caixa retangular composta por quatro áreas principais:

Explicação

Camada	Descrição
Content	Área onde o conteúdo (texto, imagem, etc.) aparece.
Padding	Espaço entre o conteúdo e a borda. É transparente.
Border	A borda que envolve o padding e o conteúdo.
Margin	Espaço externo à borda, separando o elemento de outros ao redor.

Estrutura



Modelo de caixa

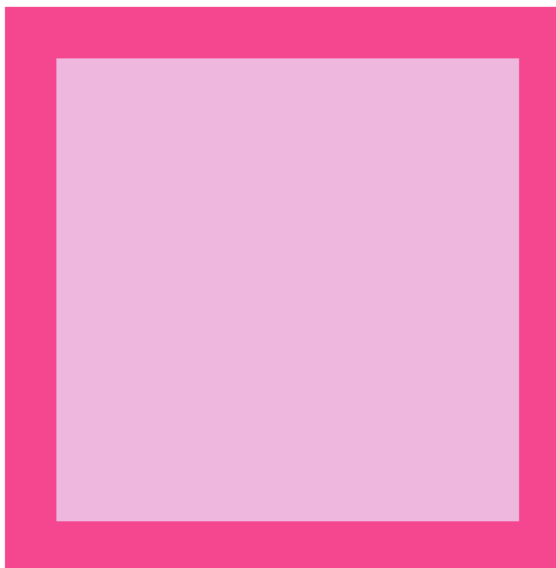
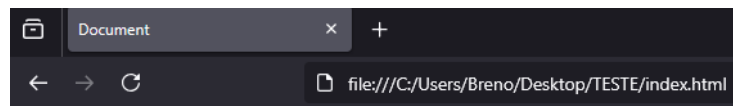
CSS

Exemplo de código

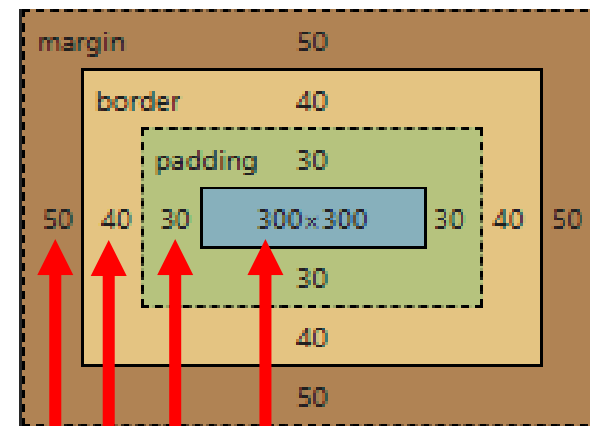
```
.container{  
  width: 300px;  
  height: 300px;  
  padding: 30px;  
  border: 40px solid #d9534f;  
  margin: 50px;  
  background-color: #f4cccc;  
}
```

```
<div class="container"></div>
```

Resultado



Estrutura do elemento



Tamanho do elemento

Tamanho do padding

Tamanho do border

Tamanho da margin

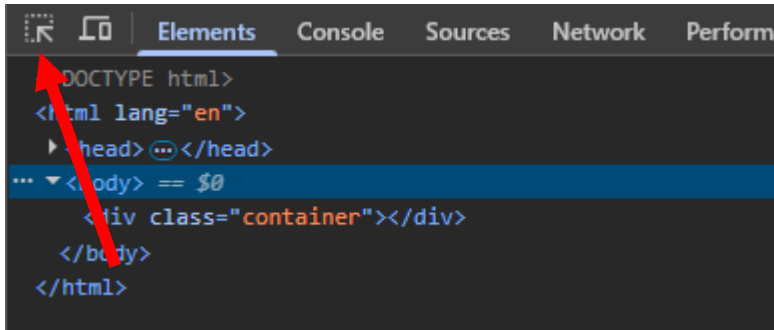
Todos os elementos HTML têm seu próprio modelo de caixa, inclusive as tags que não possuem fechamento explícito.

Modelo de caixa

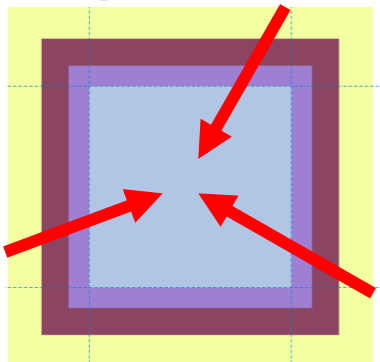
Como verificar modelos de caixa usando o navegador

O Google foi usado nesse exemplo, outros navegadores podem ser diferentes.

1. Botão direito do mouse em uma página
2. Clique na Opção “Inspecionar elemento”/“Inspecionar ou CTRL+SHIFT+C
3. Clique no seletor



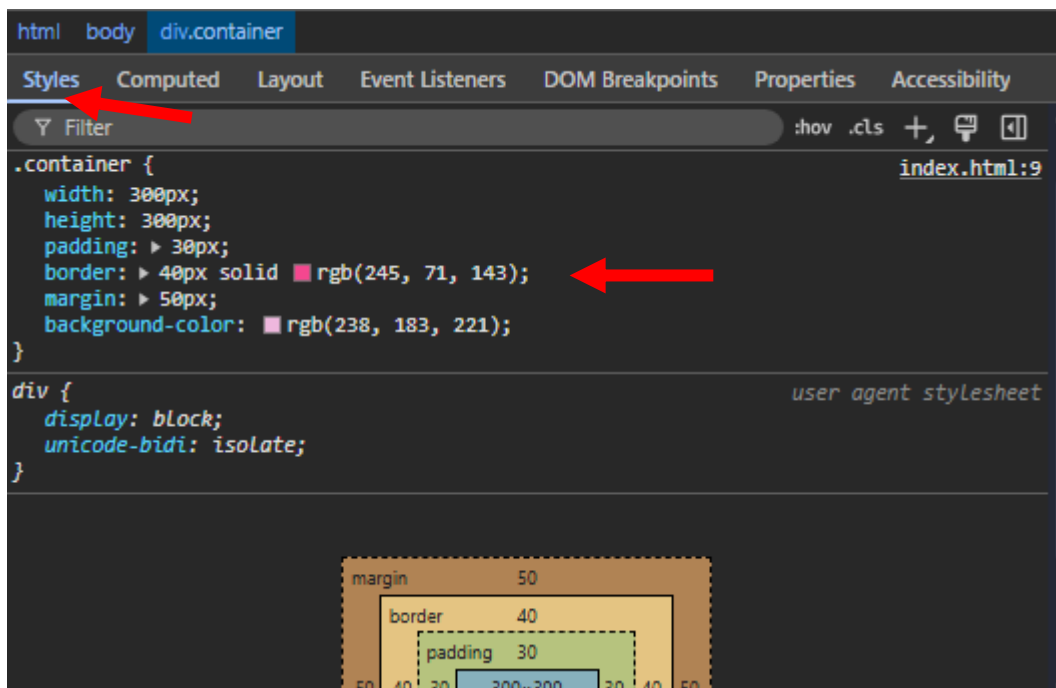
4. Clique no conteúdo do elemento (centro do container)



Modelo de caixa

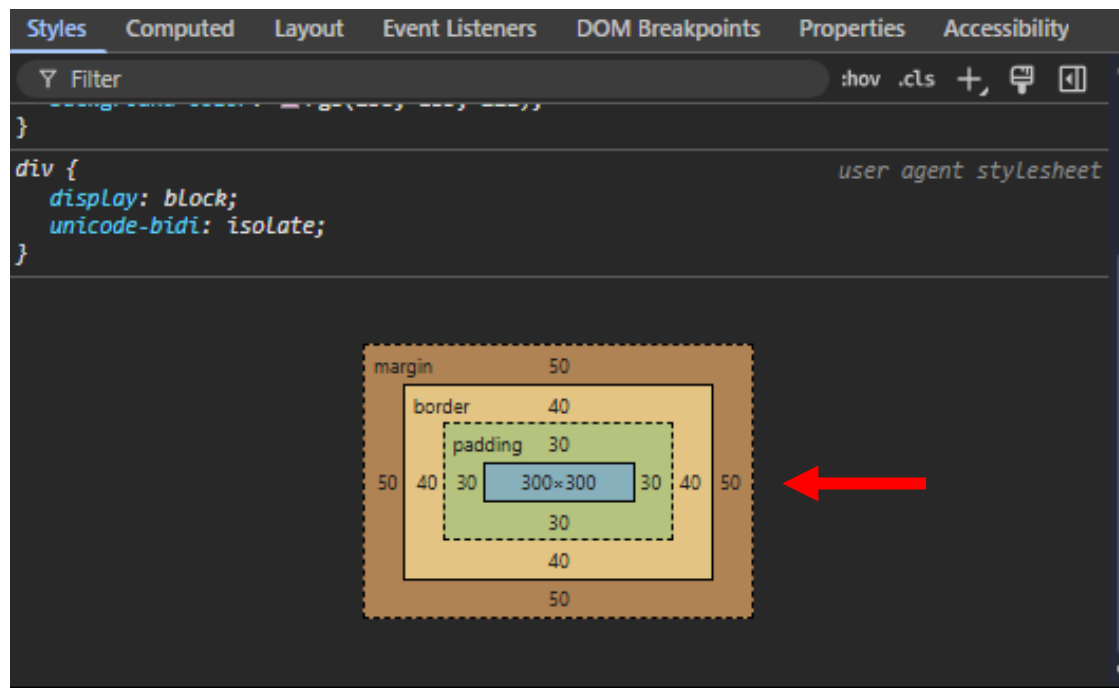
Como verificar modelos de caixa usando o navegador

5.1 Clique em Styles



Você poderá alterar os valores do CSS e ver possíveis mudanças antes de alterar o código apenas clicando e mudando os valores.

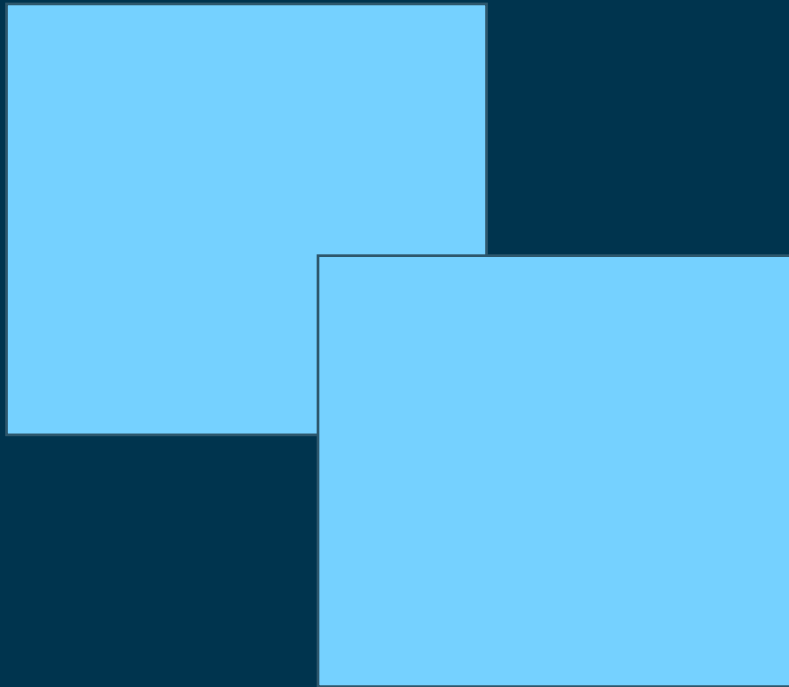
5.2 Role até o modelo de caixa



Colocando o mouse em cima das camadas é possível ver quais camadas seu código tem e onde estão cada uma delas.

Dimensionamento e posizionamento

Cascading Style Sheets



Dimensionamento e posicionamento

CSS

Quando você entende o **modelo de caixa do CSS**, ganha controle total sobre **como os elementos ocupam espaço** e **onde eles aparecem** na página. Isso envolve dois aspectos principais:

1. **Dimensionamento (Tamanho do Elemento)**
2. **Posicionamento (Onde o Elemento Aparece)**

Dimensionamento e posicionamento

CSS

Dimensionamento (Tamanho do Elemento)

Você define o tamanho total de um elemento usando:

- width e height: definem a área de conteúdo.
- padding: adiciona espaço interno ao redor do conteúdo.
- border: adiciona uma borda visível ao redor do padding.
- margin: adiciona espaço externo entre o elemento e os outros.

```
.box {  
  width: 200px;  
  padding: 20px;  
  border: 5px solid #333;  
  margin: 10px;  
}
```

O tamanho total horizontal será:

$200 \text{ (width)} + 20 \text{ (padding)} + 5 \text{ (border)} + 10 \text{ (margin)} = 235\text{px}$

Dimensionamento e posicionamento

POSITION

Posicionamento (Onde o Elemento Aparece)

Você controla a posição do elemento na página com:

position: define o tipo de posicionamento:

- static (padrão)
- relative (posiciona em relação à posição original)
- absolute (posiciona em relação ao elemento pai com position: relative)
- fixed (fica fixo na tela, mesmo com rolagem)
- sticky (gruda no topo ao rolar)

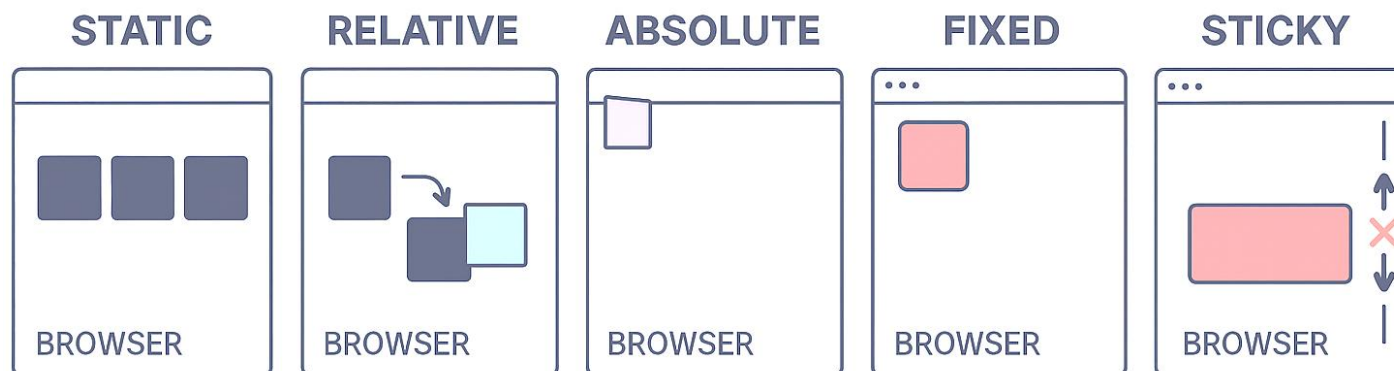
Dimensionamento e posicionamento

POSITION

STATIC

Position: static é o posicionamento padrão. O elemento aparece na ordem natural da página e não reage a comandos como top, left, right ou bottom.

`<div style="position: static;">` Elemento estático`</div>`



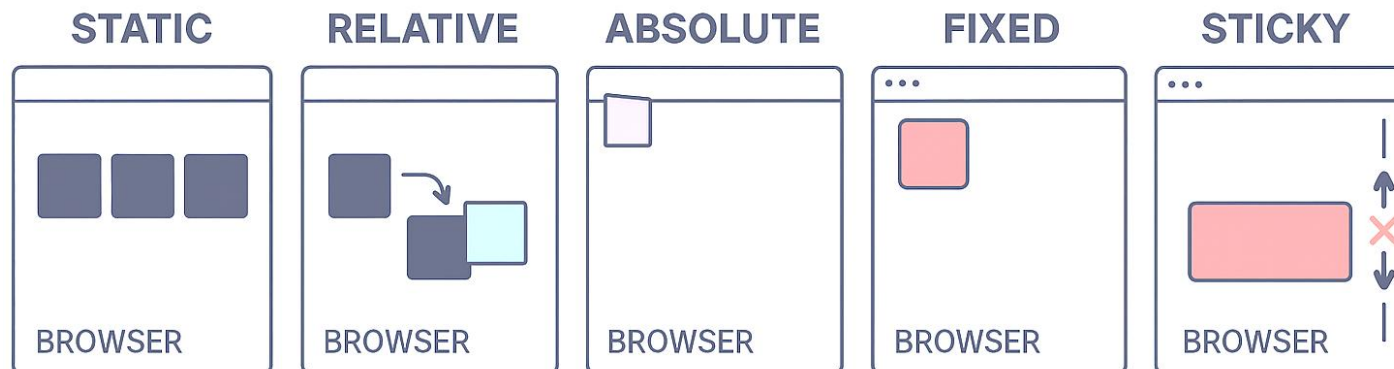
Cascading Style Sheets

POSITION

RELATIVE

Position: relative permite mover o elemento em relação ao seu lugar original. Ele continua ocupando o mesmo espaço na página, mas aparece visualmente deslocado.

`<div style="position: relative; top: 20px; left: 30px;"> Elemento relativo</div>`



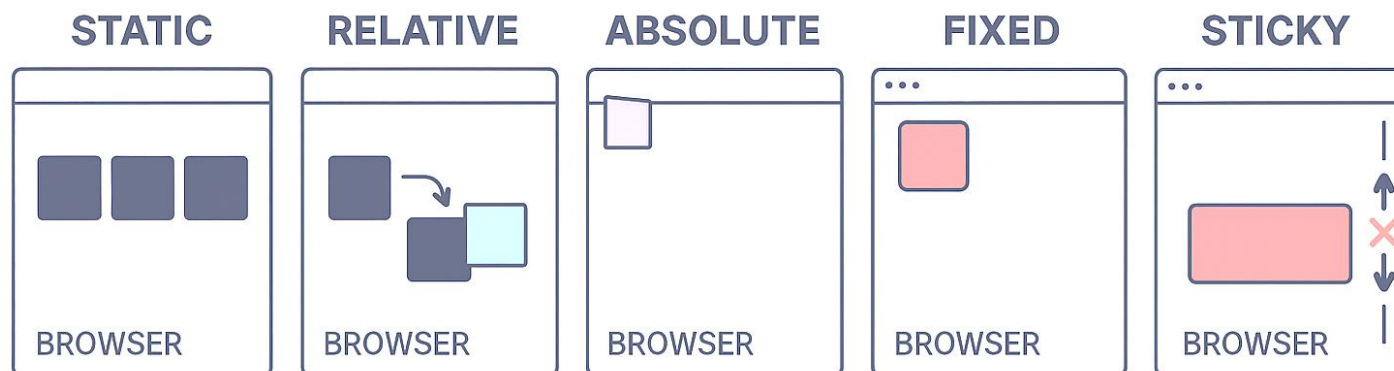
Cascading Style Sheets

POSITION

ABSOLUTE

Position: absolute tira o elemento do fluxo normal da página e o posiciona em relação ao primeiro elemento pai que tenha position: relative. Se não houver esse pai, ele se posiciona em relação ao corpo da página.

`<div style="position: relative;"> <div style="position: absolute; top: 10px; left: 10px;"> Elemento absoluto </div> </div>`



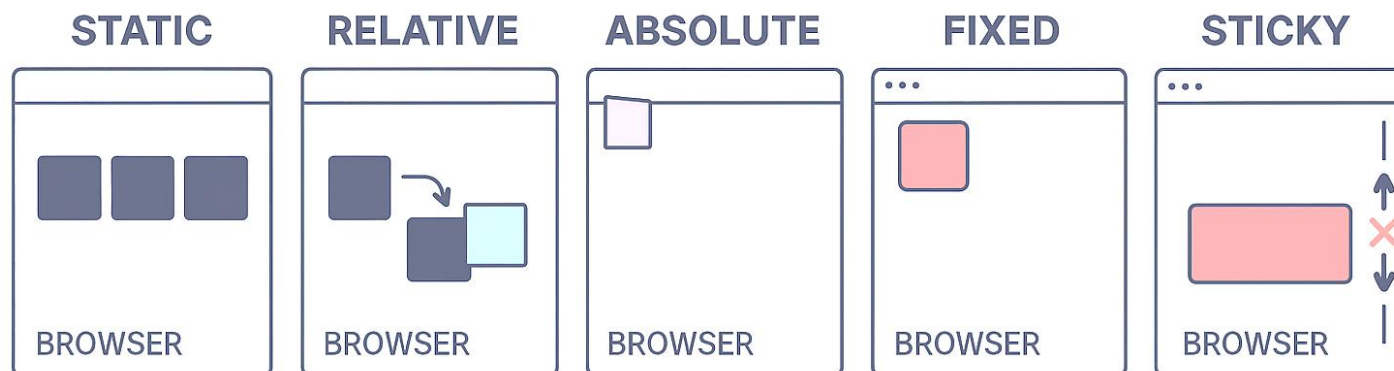
Cascading Style Sheets

POSITION

FIXED

Position: fixed fixa o elemento na tela, independente da rolagem. Ele sempre aparece no mesmo lugar, como um botão flutuante ou um cabeçalho fixo.

```
<div style="position: fixed; top: 0; left: 0; background: red;"> Cabeçalho fixo</div>
```



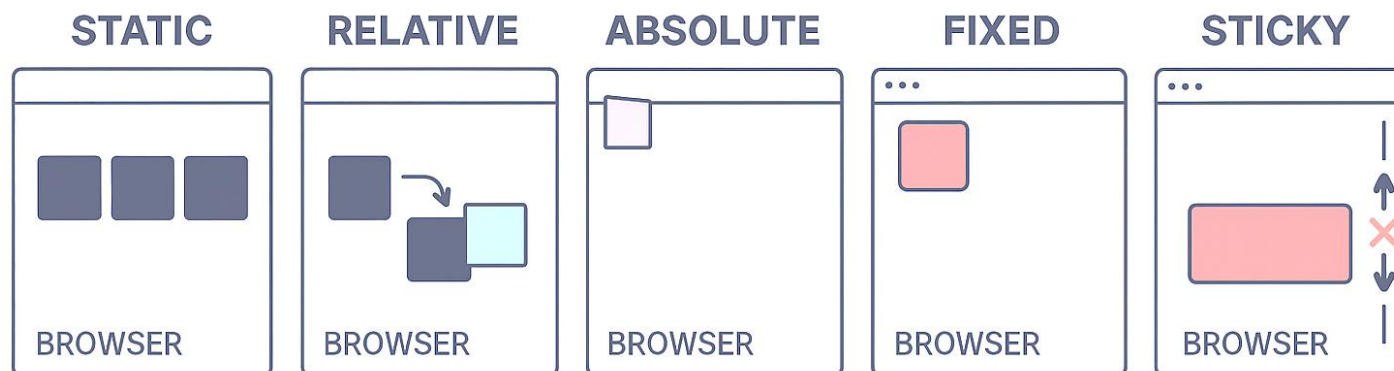
Cascading Style Sheets

POSITION

STICKY

Position: sticky combina comportamento de static e fixed. O elemento começa como estático, mas quando você rola a página e ele encosta no topo, ele gruda ali e permanece visível.

```
<div style="position: sticky; top: 0; background: yellow;"> Menu sticky</div>
```



Flex-box

Caixa Flexível



FlexBox

Caixa Flexível

Flexbox é uma forma de organizar elementos numa página da web usando CSS. Ele ajuda a colocar os itens no lugar certo, mesmo que eles tenham tamanhos diferentes ou que a tela mude de tamanho (como em celulares e computadores).

Pense num caixa de brinquedos: você pode colocar carrinhos, bonecos, bolas... cada um com um tamanho diferente. O Flexbox é como um organizador mágico que arruma tudo direitinho dentro da caixa, sem bagunça.

Funcionamento:

- **O container (caixa)** — é o elemento que vai conter os itens.
- **Os itens (objetos)** — são os elementos que você quer organizar dentro da caixa.

Flexbox

Display

A propriedade **Display** no CSS define como um elemento aparece na página, se ele ocupa uma linha inteira, se fica ao lado de outros, se desaparece, etc. É como dizer: “Esse container vai ficar em pé sozinho” ou “Esse vai se encaixar ao lado dos outros”.

Tipos de Display:

- Block
- None
- Flex
- Grid
- Inline
- Inline-block

Código

```
Display: block;  
Display: none;  
Display: flex;  
Display: grid;  
Display: inline;  
Display: inline-block;
```


Flexbox

Display

Display block faz com que o elemento ocupe toda a largura disponível e quebre linha antes e depois. Ele respeita propriedades como largura, altura, margem e padding, sendo ideal para estruturar seções da página, como div, section ou p.

Display inline mantém os elementos na mesma linha, como palavras em um texto. Ele ignora largura e altura definidas, mas aceita margens e padding horizontais. É usado para elementos como span ou a.

Display inline-block combina o comportamento inline com a capacidade de respeitar largura, altura, margem e padding. Os elementos ficam lado a lado, mas podem ser dimensionados e estilizados como blocos. Ideal para botões ou caixas ajustáveis.



Flexbox

Display

Display none remove completamente o elemento da renderização. Ele não aparece na tela e não ocupa espaço, sendo útil para esconder elementos dinamicamente com CSS ou JavaScript.

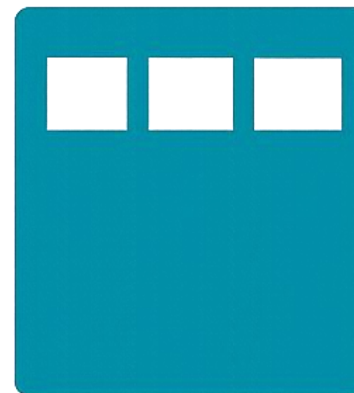
Display flex transforma o elemento em um container flexível, permitindo alinhar e distribuir os itens filhos em linha ou coluna. Oferece controle sobre espaçamento, ordem e alinhamento dos elementos, sendo útil para layouts responsivos.

Display grid cria um sistema de linhas e colunas, permitindo posicionar os elementos filhos com precisão. É ideal para layouts mais complexos e estruturados, como páginas com cabeçalho, barra lateral e conteúdo principal.

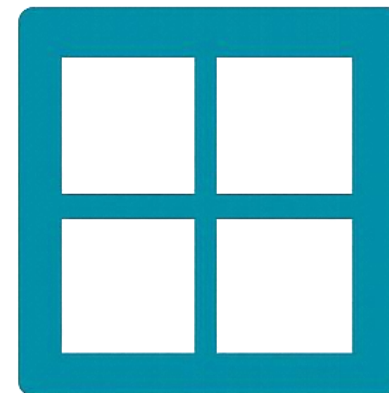
None



Flex



Grid



Flexbox

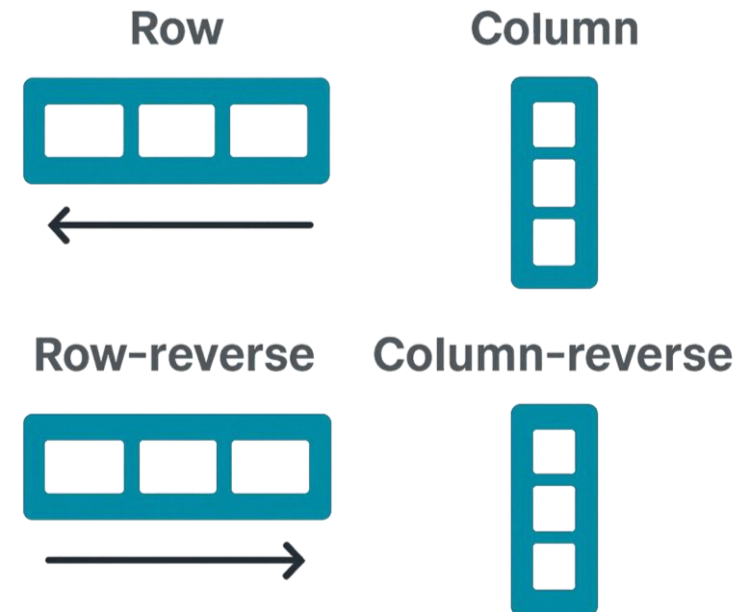
flex-direction

Essas propriedades controlam como os itens dentro de um container com **display:flex** se organizam e se comportam.

Ex: <div> possui **display: flex** e o <p> dentro da div possui **flex-Direction: row**.

- **row:** em linha, da esquerda pra direita.
- **column:** em coluna, de cima pra baixo.
- **Row-reverse:** em linha, mas da direita pra esquerda.
- **Column-reverse:** em coluna, mas de baixo pra cima.

flex-direction: row;
flex-direction: column;
flex-direction: row-reverse;
flex-direction: column-reverse;



Flexbox

Outros

Propriedades do Container

- `flex-direction`: Define a direção dos itens (`row`, `row-reverse`, `column`, `column-reverse`).
- `justify-content`: Alinha os itens no eixo principal (`flex-start`, `center`, `space-between`, etc.).
- `align-items`: Alinha os itens no eixo cruzado (`stretch`, `center`, `flex-start`, etc.).
- `flex-wrap`: Permite que os itens quebrem linha (`nowrap`, `wrap`, `wrap-reverse`).

Propriedades dos Itens

- `flex-grow`: Define quanto o item deve crescer em relação aos outros.
- `flex-shrink`: Define quanto o item deve encolher.
- `flex-basis`: Define o tamanho inicial do item antes da distribuição do espaço.
- `order`: Altera a ordem visual dos itens.

Flexbox

justify-content

Controla o **alinhamento dos itens no eixo principal** (horizontal se for row, vertical se for column).

- **flex-start:** alinhado ao começo.
- **Flex-end:** alinhado ao fim.
- **center:** centralizado .
- **space-between:** espaço entre os itens.
- **space-around:** espaço ao redor de cada item.
- **space-evenly:** espaço igual entre todos.

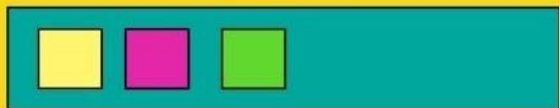
```
justify-content : flex-start  
justify-content : flex-end  
justify-content : center  
justify-content : space-between  
justify-content : space-around  
justify-content : space-between  
justify-content : space-evenly
```

Flexbox

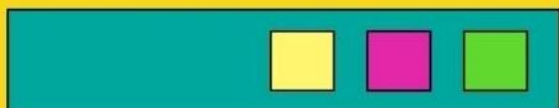
justify-content

CSS Flexbox Justify-Content

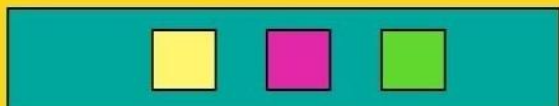
flex-start



flex-end



centre



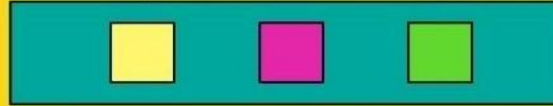
space-around



space-between



space-evenly



justify-content : flex-start
justify-content : flex-end
justify-content : center
justify-content : space-between
justify-content : space-around
justify-content : space-between
justify-content : space-evenly

Flexbox

align-items

Alinha os itens no **eixo cruzado** (perpendicular ao principal).

- **stretch:** estica os itens para ocupar o espaço.
- **center:** centraliza.
- **flex-start:** alinha no topo.
- **flex-end:** alinha embaixo.

Aligns-itens: stretch

Aligns-itens: center

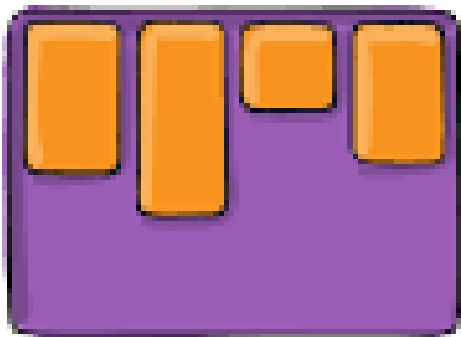
Aligns-itens: flex-start

Aligns-itens: flex-end

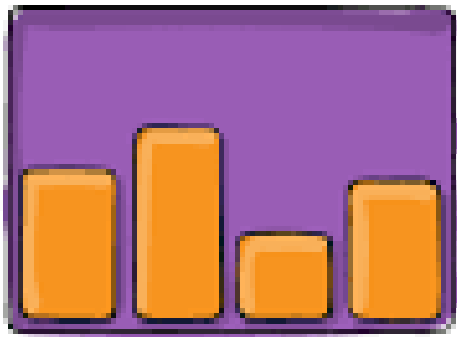
Flexbox

align-items

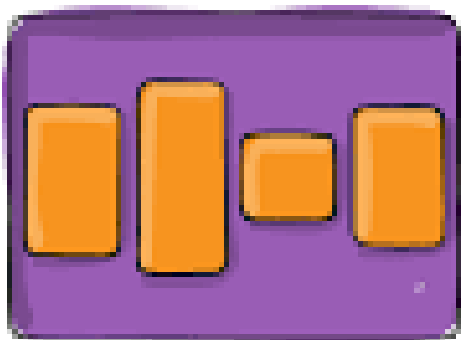
flex-start



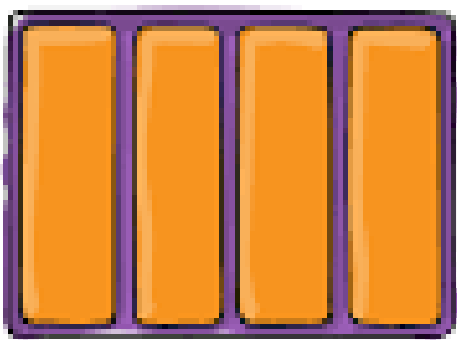
flex-end



center



stretch



Aligns-items: stretch

Aligns-items: center

Aligns-items: flex-start

Aligns-items: flex-end

Flexbox

flex-wrap

Define se os itens **podem quebrar linha** quando não cabem.

- **nowrap:** tudo numa linha só (pode ficar apertado).
- **wrap:** quebra linha quando necessário.
- **wrap-reverse:** quebra linha, mas de baixo pra cima.

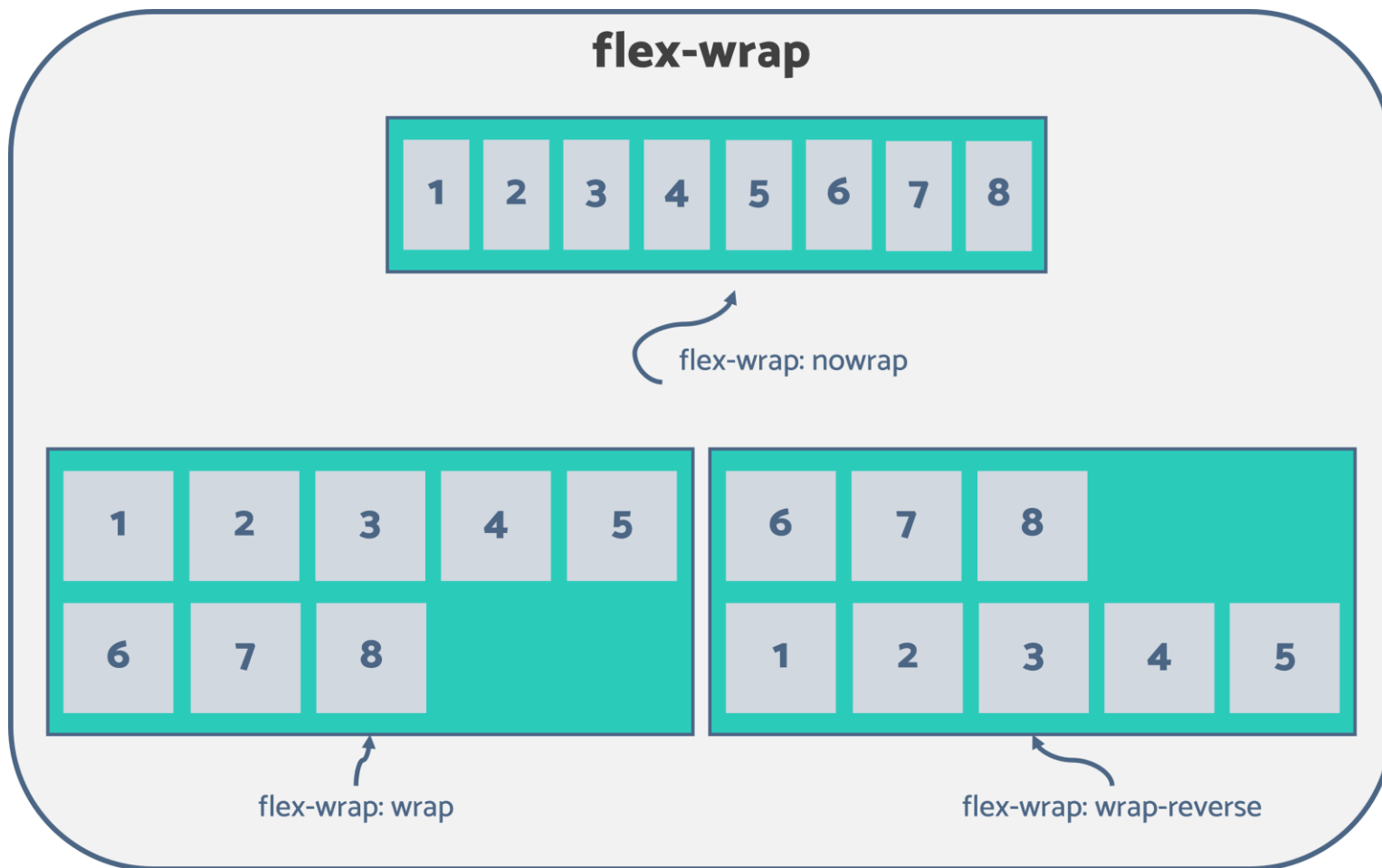
flex-wrap: nowrap

flex-wrap: wrap

flex-wrap: wrap-reverse

Flexbox

flex-wrap



flex-wrap: nowrap
flex-wrap: wrap
flex-wrap: wrap-reverse

Flexbox

Joguinho

<https://flexboxfroggy.com/>

FLEXBOX FROGGY

◀ Nível 22 de 24 ▶

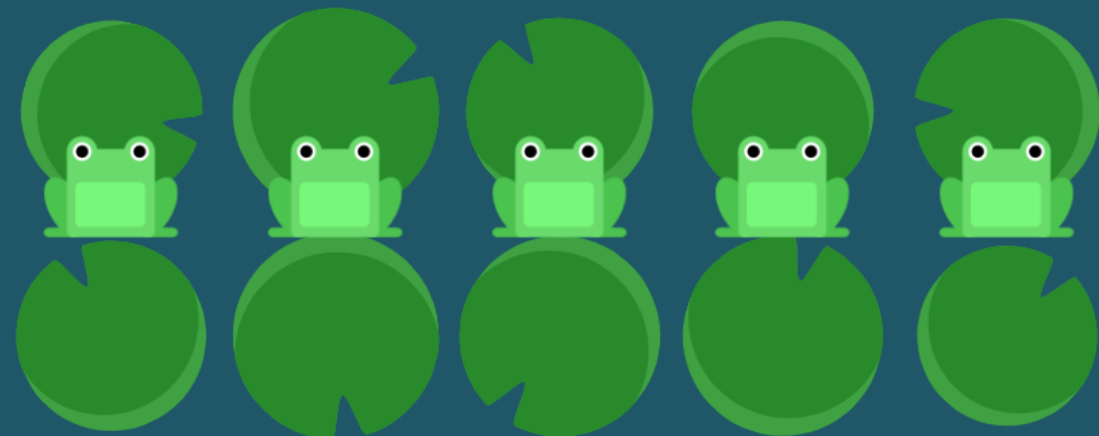
Agora a correnteza agrupou as vitórias-régias no fundo. Use `align-content` para guiar os sapos até lá.

```
1 #pond {  
2   display: flex;  
3   flex-wrap: wrap;  
4  
5 }
```

Próximo

Flexbox Froggy foi criado por [Codepip](#) • [YouTube](#) • [Twitter](#) • [GitHub](#) • [Configurações](#)

Quer aprender CSS grid? Jogue [Grid Garden](#) or [Anchoreum](#).



Outros tópicos

Fundamentos

1. Unidades de medida e valores

px, %, em, rem, vh, vw — como e quando usar cada uma para garantir flexibilidade e precisão no layout.

2. Fontes e tipografia

Estilização de texto com propriedades como font-family, font-size, font-weight, line-height, text-align, entre outras.

3. Pseudo-classes e pseudo-elementos

Estilos condicionais e decorativos usando :hover, :focus, :nth-child, ::before, ::after, etc.

4. Media Queries e responsividade

Adaptação de estilos para diferentes tamanhos de tela usando @media e breakpoints.

5. Transições e animações

Criação de efeitos visuais com transition, animation e @keyframes para melhorar a experiência do usuário.

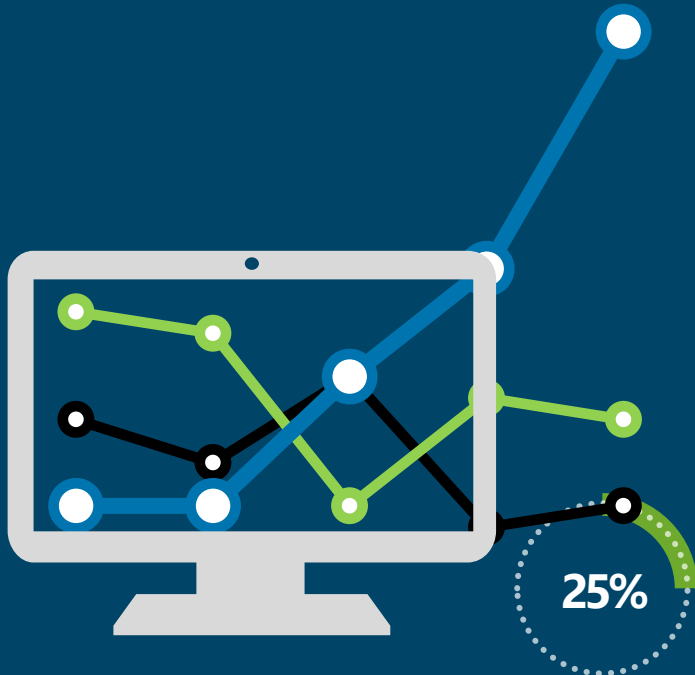
6. Variáveis CSS e organização

Uso de variáveis com --nome e var(), além de boas práticas para manter o código limpo e reutilizável.

7. Especificidade e herança

Entendimento de como o CSS decide qual estilo aplicar, como funciona a cascata, herança e o uso de !important.

Muito obrigado!



Prof. M. Sc. Felipe Ivo da Silva
felipe_ivodasilva@hotmail.com
(16) 98805-0832